

Sualeh 10303, Burair 09628

TASK 1:

CODE:

```
=====
* TASK 1 — Standard Dynamic Memory Allocation (malloc/calloc/free)
* -----
* Demonstrates heap allocation using standard C library functions.
* Uncomment the block below to run Task 1 instead of Task 2.
* ===== */

/*
uart_send("=====\r\n");
uart_send(" Lab 10 Task 1: Dynamic Memory Alloc \r\n");
uart_send("=====\r\n\r\n");

// -----
// STEP 1 — malloc()
// Allocates N integers (uninitialized).
// Each element is assigned the value i * 2.
// -----
uart_send("[STEP 1] malloc() — allocate and initialize\r\n");

int *arr_malloc = (int *)malloc(N * sizeof(int));

if (arr_malloc == NULL)
{
    uart_send(" ERROR: malloc() returned NULL. Heap exhausted!\r\n");
}
else
{
    uart_send(" malloc() successful.\r\n");

    for (int i = 0; i < N; i++)
        arr_malloc[i] = i * 2;

    uart_send(" Contents (val = i*2):\r\n");
    for (int i = 0; i < N; i++)
        uart_send_int("arr_malloc", i, arr_malloc[i]);
}

// -----
// STEP 2 — calloc()
// Allocates N integers, all zero-initialized by calloc().
// Verifies zeros first, then assigns i + 1 to each element.
// -----
uart_send("\r\n[STEP 2] calloc() — zero-init, then assign\r\n");

int *arr_calloc = (int *)calloc(N, sizeof(int));

if (arr_calloc == NULL)
{
    uart_send(" ERROR: calloc() returned NULL. Heap exhausted!\r\n");
}
else
{
    uart_send(" calloc() successful.\r\n");

    uart_send(" Before assignment (expect all 0):\r\n");
```

```

for (int i = 0; i < N; i++)
    uart_send_int("arr_malloc", i, arr_malloc[i]);

for (int i = 0; i < N; i++)
    arr_malloc[i] = i + 1;

uart_send(" After assignment (val = i+1):\r\n");
for (int i = 0; i < N; i++)
    uart_send_int("arr_malloc", i, arr_malloc[i]);
}

// -----
// STEP 3 — free()
// Releases both arrays back to the heap.
// Pointers set to NULL to prevent dangling references.
// -----
uart_send("\r\n[STEP 3] free() — release heap memory\r\n");

free(arr_malloc);
arr_malloc = NULL;
uart_send(" arr_malloc freed. Pointer set to NULL.\r\n");

free(arr_malloc);
arr_malloc = NULL;
uart_send(" arr_malloc freed. Pointer set to NULL.\r\n");

uart_send("\r\n===== \r\n");
uart_send(" Task 1 Complete. All memory released. \r\n");
uart_send("===== \r\n");
*/

/* =====
* TASK 2 — Custom Heap Driver (Direct SRAM Addressing)
* -----
* Uses a custom allocator (Lab10_heap_driver.c) that manages a fixed
* 4 KB region of SRAM starting at 0x20001000, divided into 256 blocks
* of 16 bytes each, tracked via a bitmap (block_map[]).
* ===== */

```

```
51     MX_GPIO_Init();
52     MX_I2C1_Init();
53     MX_SPI1_Init();
54     MX_USART2_UART_Init();
55     MY_HCD_BCD_Init();

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Memory successfully freed. Pointers cleared.
=== Task 1 Complete ===
=== Dynamic Allocation Demo ===

>> malloc()
malloc ok
Data: 0, 2, 4, 6, 8, 10, 12, 14, 16, 18

>> calloc()
calloc ok
Initial (zero): 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
After assign: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

>> free()
m_ptr cleared
c_ptr cleared

All done.
```

TASK 2:

DRIVER CODE:

```
#define HEAP_START_ADDR ((uint8_t*)0x20001000)
#define HEAP_SIZE (4 * 1024)
#define BLOCK_SIZE 16
#define BLOCK_COUNT (HEAP_SIZE / BLOCK_SIZE)

/* Block map: 0 = free, 1 = used */
static uint8_t block_map[BLOCK_COUNT];

/* ===== heap_init ===== */
void heap_init(void)
{
    // Set all blocks to free
    memset(block_map, 0, sizeof(block_map));
}

/* ===== heap_alloc ===== */
void* heap_alloc(size_t size)
{
    if (size == 0)
        return NULL;

    // Calculate required number of blocks
    size_t needed = (size + BLOCK_SIZE - 1) / BLOCK_SIZE;

    size_t count = 0;
```

```

size_t start = 0;

// Search for contiguous free blocks
for (size_t i = 0; i < BLOCK_COUNT; i++)
{
    if (block_map[i] == 0)
    {
        if (count == 0)
            start = i;

        count++;

        if (count == needed)
        {
            // Mark blocks as used
            for (size_t j = start; j < start + needed; j++)
            {
                block_map[j] = 1;
            }

            // Return pointer to start address
            return (void*)(HEAP_START_ADDR + (start * BLOCK_SIZE));
        }
    }
    else
    {
        count = 0; // reset if block is occupied
    }
}

// No space available
return NULL;
}

/* ===== heap_free ===== */
void heap_free(void* ptr)
{
    if (ptr == NULL)
        return;

    uint8_t* addr = (uint8_t*)ptr;

    // Check if pointer is within heap range
    if (addr < HEAP_START_ADDR || addr >= (HEAP_START_ADDR + HEAP_SIZE))
        return;

    // Check if pointer is aligned to block boundary
    if ((addr - HEAP_START_ADDR) % BLOCK_SIZE != 0)
        return;

    // Calculate block index
    size_t index = (addr - HEAP_START_ADDR) / BLOCK_SIZE;

    // Free blocks until a free block is found
    for (size_t i = index; i < BLOCK_COUNT; i++)
    {
        if (block_map[i] == 1)
        {
            block_map[i] = 0;
        }
    }
    else

```

```

{
break; // stop when already free
}
}
}

```

```

=== Custom Heap Driver (Direct SRAM) ===

[INFO] Heap initialized.
[INFO] Both blocks allocated successfully.

[ADDR] block1 = 0x20001000
[ADDR] block2 = 0x20001020

[DATA] block1 contains: Data in Block 1
[DATA] block2 contains: Text from Block 2

[INFO] Freeing block1...
[INFO] Freeing block2...
Blocks freed.

=====
Task 2 Complete.
=====

```

EVALUATION QUESTIONS:

- (a) Using malloc() without checking for NULL is risky because if the system runs out of heap memory, malloc() will return NULL. If the program still tries to use that pointer (like storing values in the array), it will access invalid memory and can crash or behave unpredictably. In our code, we handled this properly by checking if the pointer is NULL and printing an error message, which prevents such issues.
- (b) The allocation simply fails. In standard C functions like malloc() and calloc(), this means they return NULL. Similarly, in our custom heap driver, the heap_alloc() function returns NULL if there are not enough free contiguous blocks. This ensures that no invalid memory is assigned, but the program must check for this condition to avoid errors.
- (c) Memory leaks can be detected by carefully tracking whether all allocated memory is properly freed. In our code, we used free() and also set pointers to NULL, which is good practice. In the custom heap system, we can monitor the block_map[] array to see if any blocks remain marked as used after execution. If memory is allocated but not freed, over time the system will run out of available memory, and allocation functions will start failing. Adding debug prints and checking heap usage regularly can help identify such issues.